



Lista no. 2:

Manejo de arreglos usando Numpy - Python

Inteligencia Artificial

Profra. Vanessa Alejandra Camacho Vázquez

Grupo: **6CM3**

Nombres de los integrantes:

- Aguirre Lanto Víctor Manuel
- Gasca Fragoso Pedro
- Guevara Badillo Areli Alejandra
- Montiel Toro Arael de Jesús
- Ramírez Lozano Gael Martin

Enlace Notebook: [Lista 2: Manejo de arreglos usando Numpy - Python](#)

Viernes, 19 de septiembre, 2025

CONTENIDO

CREACIÓN BÁSICA DE ARRAYS.....	3
INDEXACIÓN BÁSICA.....	4
DIFERENTES TIPOS DE DATOS EN LOS ARRAYS	4
TAMAÑO DE LOS ARRAYS.....	5
CAMBIO DE FORMA DE LOS ARREGLOS.....	6
TRANSPUESTA.....	7
RECORRER LOS VALORES DE UN ARREGLO	7
GENERACIÓN DE ARREGLOS	9
OPERACIONES CON ARRAYS	11
MULTIPLICACIÓN MATRICIAL	16



Creación básica de arrays

```
# Importar la biblioteca NumPy
import numpy as np

# Crea un arreglo de 0 dimensiones, que es básicamente un escalar.
a = np.array(13) #array de dimensión 0
print("type(a):", type(a), "\n")
print("a:", a, sep = "\n")
a
```

type(a): <class 'numpy.ndarray'>

a:
13
array(13)

```
(61) # Crea un arreglo de 1 dimensión, similar a una lista de números.
✓ Os b = np.array([1, 2, 3, 4]) #array de dimensión 1
print("b:", b, sep = "\n")
b

b:
[1 2 3 4]
array([1, 2, 3, 4])

(62) # Crea un arreglo de 2 dimensiones, similar a una matriz o tabla.
✓ Os c = np.array([[1, 2, 3, 4], [5, 6, 7, 8]]) #array de dimensión 2
print("c:", c, sep = "\n")
c

c:
[[1 2 3 4]
 [5 6 7 8]]
array([[1, 2, 3, 4],
       [5, 6, 7, 8]])

(63) # Crea un arreglo de 3 dimensiones, como un cubo de números.
✓ Os d = np.array([[[1, 2, 3, 4], [5, 6, 7, 8]], [[9, 10, 11, 12], [13, 14, 15, 16]]]) #array de dimensión 3
print("d:", d, sep = "\n")
d

d:
[[[ 1  2  3  4]
  [ 5  6  7  8]]
 [[ 9 10 11 12]
  [13 14 15 16]]]
array([[[1, 2, 3, 4],
        [5, 6, 7, 8]],
       [[9, 10, 11, 12],
        [13, 14, 15, 16]]])
```

```
# Imprime el número de dimensiones de cada arreglo creado.
print(a.ndim)
print(b.ndim)
print(c.ndim)
print(d.ndim)
```

0
1
2
3

EXPLICACIÓN:

El código importa la biblioteca **NumPy** y crea arreglos de diferentes dimensiones.

Primero se define un arreglo de 0 dimensiones con un único valor (a).

Luego se crea un arreglo de 1 dimensión con una lista de números (b).

Después se genera un arreglo de 2 dimensiones en forma de matriz con filas y columnas (c).

Finalmente, se construye un arreglo de 3 dimensiones que representa un conjunto de matrices, similar a un cubo (d).

En cada caso se imprime el contenido del arreglo.

EXPLICACIÓN:

Ese fragmento del código imprime el número de dimensiones que tiene cada arreglo creado anteriormente.

- a.ndim muestra que a tiene **0 dimensiones**.
- b.ndim muestra que b tiene **1 dimensión**.
- c.ndim muestra que c tiene **2 dimensiones**.
- d.ndim muestra que d tiene **3 dimensiones**.

Indexación básica

```
# Muestra cómo acceder a elementos individuales o filas/capas en arreglos.
print("a =", a)
print("b[2] =", b[2])
print("c[1] =", c[1])
print("d[2] =", d[1])

a = 13
b[2] = 3
c[1] = [5 6 7 8]
d[2] = [[ 9 10 11 12]
        [13 14 15 16]]

# Muestra cómo acceder a elementos específicos en arreglos multidimensionales.
print("a =", a)
print("b[2] =", b[2])
print("c[1,2] =", c[1,2])
print("d[1,0,2] =", d[1,0,2])

a = 13
b[2] = 3
c[1,2] = 7
d[1,0,2] = 11

# Muestra cómo seleccionar rangos de elementos (slicing) en arreglos.
print(b[1:4]) #b = [1 2 3 4]
print(c[1:3]) #c = [[1 2 3 4][5 6 7 8]]
print(d[1:3,1:3,0:2])

[2 3 4]
[6 7]
[[13 14]]
```

EXPLICACIÓN:

Aquí trabajamos con los arreglos de **NumPy** para acceder a su información.

Primero se enseña cómo obtener elementos completos, como filas en una matriz o capas en un arreglo tridimensional.

Después se demuestra cómo acceder a posiciones específicas dentro de arreglos multidimensionales indicando los índices correspondientes.

Finalmente, se utiliza **slicing** para seleccionar rangos de elementos en diferentes dimensiones, lo que permite extraer partes de los arreglos en lugar de un solo valor.

El código explica las formas básicas de indexar y manipular datos dentro de arreglos de una o varias dimensiones.

Diferentes tipos de datos en los arrays

```
# Muestra el tipo de dato de los elementos dentro de un arreglo.
frutas = np.array(['Manzana', 'Naranja', 'Uva']) #cadenas
print(frutas.dtype)
```

```
<U7
```

Se crea un arreglo de cadenas de texto y se muestra que los elementos son de tipo **string** utilizando la propiedad **.dtype** de NumPy.

```
# Convierte un arreglo de números a un arreglo de cadenas.
number_to_string = np.array([6, 1, 2, 4, 623, 8], dtype=str) #b denota un string
print(number_to_string.dtype)
print(number_to_string)
```

```
<U3
['6' '1' '2' '4' '623' '8']
```

Se convierte un arreglo de números a un arreglo de **cadenas de texto**, lo que cambia el tipo de dato de los elementos a **string**, y luego se muestra el contenido del nuevo arreglo.

```
# Intenta crear un arreglo con tipos de datos que no se pueden convertir (causa un error).
try:
    error_array = np.array(['a', '2', '3'], dtype=int)
except:
    print("hay un error en su lógica")
```

hay un error en su lógica

Se intenta crear un arreglo de tipo **entero** a partir de elementos que no se pueden convertir (como la letra 'a'), lo que provoca un error. Para manejarlo, se utiliza un bloque **try-except** que captura el error y muestra un mensaje indicando que hay un problema con los datos.

```
# Convierte un arreglo de números con decimales (flotantes) a un arreglo de números enteros.
floating_array = np.array([1.1, 2.1, 3.1]) #punto flotante

int_array = floating_array.astype(int)
print(int_array)
```

[1 2 3]

Se convierte un arreglo de números **decimales (flotantes)** a un arreglo de **enteros** usando el método `.astype(int)`, lo que descarta la parte decimal de cada elemento y deja solo la parte entera.

```
# Muestra cómo especificar diferentes tamaños para los números enteros en un arreglo.
vector = np.array([127, -127, 32767, -32767], dtype=np.int32) # entero de 1 byte = 8 bits (1 bit se necesita para almacenar el signo)
print("vector:\t", vector)
print("vector.dtype: ", vector.dtype)
vector = np.array([127, -127, 32767, -32767], dtype=np.int16) # entero de 2 bytes = 16 bits (1 bit se necesita para almacenar el signo)
print("vector:\t", vector)
print("vector.dtype: ", vector.dtype)
```

```
vector: [ 127 -127 32767 -32767]
vector.dtype: int32
vector: [ 127 -127 32767 -32767]
vector.dtype: int16
```

Se muestra cómo crear arreglos de enteros especificando **diferentes tamaños de almacenamiento** mediante `dtype`. Al usar `np.int32` los números se guardan en 32 bits, y con `np.int16` se usan 16 bits. Esto permite controlar la memoria que ocupa el arreglo y la precisión de los números enteros.

Tamaño de los arrays

```
# Imprime la forma (el número de elementos en cada dimensión) de cada arreglo.
print(a.shape)
print(b.shape)
print(c.shape)
print(d.shape)
```

()
(4,)
(2, 4)
(2, 2, 4)

Se muestra la **forma de cada arreglo**, es decir, el número de elementos que tiene en cada dimensión. La propiedad `.shape` permite ver la estructura de los arreglos, desde un solo valor hasta matrices y arreglos tridimensionales.

Cambio de forma de los arreglos

```
# Cambia la forma de un arreglo 2D a un arreglo 1D.
print("c =", c, "\n") # c = [[1 2 3 4][5 6 7 8]]
c_1D = c.reshape(1, 8)
print("c_1D =", c_1D, "\n")
```

```
→ c = [[1 2 3 4]
       [5 6 7 8]]

c_1D = [[1 2 3 4 5 6 7 8]]
```

```
# Cambia la forma de un arreglo 2D a otra forma 2D.
c_4D = c.reshape(4, 2)
print("c_4D =", c_4D, "\n")
```

```
→ c_4D = [[1 2]
          [3 4]
          [5 6]
          [7 8]]
```

```
# Cambia la forma de un arreglo 1D a 2D,
# dejando que NumPy calcule una de las dimensiones (-1).
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
arr.reshape(3, -1)
```

```
→ array([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
```

```
# Convierte un arreglo multidimensional a un arreglo 1D (aplanar).
d.reshape(-1)
```

```
→ array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16])
```

EXPLICACIÓN:

Se modifica la **forma de un arreglo** utilizando el método `.reshape()`, lo que permite reorganizar los datos en diferentes dimensiones sin alterar los valores que contiene.

Primero, un arreglo 2D se convierte en un arreglo de **una sola fila**, es decir, una estructura más lineal que agrupa todos los elementos en una fila continua.

Después, el mismo arreglo se reorganiza en otra forma 2D diferente, con más filas y menos columnas, mostrando cómo los mismos datos se pueden distribuir de distintas maneras según las necesidades de procesamiento o visualización.

EXPLICACIÓN:

NumPy permite cambiar la forma de un arreglo de manera flexible.

Un arreglo 1D puede convertirse en 2D indicando solo una de las dimensiones y dejando que NumPy calcule automáticamente la otra con `-1`.

También es posible aplanar un arreglo multidimensional a 1D, reorganizando todos sus elementos en una sola dimensión, lo que facilita trabajar con los datos en distintas estructuras.

Transpuesta

```
# Calcula la transpuesta de un arreglo 2D
# (intercambia filas por columnas).
print("c =", c, "\n")
print("c.T =", c.T)

c = [[1 2 3 4]
      [5 6 7 8]]

c.T = [[1 5]
        [2 6]
        [3 7]
        [4 8]]
```

EXPLICACIÓN:

Se calcula la **transpuesta** de un arreglo 2D, lo que intercambia sus filas por columnas.

Esto reorganiza los datos de manera que las filas se convierten en columnas y las columnas en filas, sin alterar los valores del arreglo

Recorrer los valores de un arreglo

```
# Itera sobre los elementos de un arreglo 1D.
for x in b:
    print(x, "\n")

1
2
3
4

# Itera sobre las filas de un arreglo 2D.
for x in c:
    print(x, "\n")

[1 2 3 4]
[5 6 7 8]

# Itera sobre las "capas" de un arreglo 3D.
for x in d:
    print(x, "\n")

[[1 2 3 4]
 [5 6 7 8]]

[[ 9 10 11 12]
 [13 14 15 16]]
```

EXPLICACIÓN:

Se recorre un arreglo de diferentes dimensiones para acceder a sus elementos.

En un arreglo 1D se itera sobre cada valor individual, en un arreglo 2D se recorre fila por fila, y en un arreglo 3D se itera sobre cada "capa" completa.

Esto permite procesar o visualizar los datos de manera ordenada según la dimensión del arreglo.

```
# Itera sobre todos los elementos de un arreglo 3D usando bucles anidados.
for x in d:
    for y in x:
        for z in y:
            print(z)
```

EXPLICACIÓN:

Se recorren todos los elementos de un arreglo 3D utilizando **bucles anidados**, lo que permite acceder a cada valor individual dentro de cada capa y fila.

Esto es útil para procesar o analizar cada elemento de manera detallada en arreglos multidimensionales.

```
# Itera eficientemente sobre todos los elementos de un arreglo usando np.nditer.
for x in np.nditer(d):
    print(x)
```

EXPLICACIÓN:

Se recorre de manera eficiente todos los elementos de un arreglo multidimensional utilizando **np.nditer**, lo que permite acceder a cada valor individual sin necesidad de usar bucles anidados, simplificando la iteración sobre arreglos de cualquier dimensión.

```
# Itera sobre los elementos de un arreglo y obtiene su índice usando np.ndenumerate.
for i, x in np.ndenumerate(d):
    print('i:', i, '\t\tx:', x)
```



```
i: (0, 0, 0)          x: 1
i: (0, 0, 1)          x: 2
i: (0, 0, 2)          x: 3
i: (0, 0, 3)          x: 4
i: (0, 1, 0)          x: 5
i: (0, 1, 1)          x: 6
i: (0, 1, 2)          x: 7
i: (0, 1, 3)          x: 8
i: (1, 0, 0)          x: 9
i: (1, 0, 1)          x: 10
i: (1, 0, 2)          x: 11
i: (1, 0, 3)          x: 12
i: (1, 1, 0)          x: 13
i: (1, 1, 1)          x: 14
i: (1, 1, 2)          x: 15
i: (1, 1, 3)          x: 16
```

EXPLICACIÓN:

Se recorre un arreglo multidimensional obteniendo **tanto el índice como el valor** de cada elemento mediante `np.ndenumerate`.

Esto permite conocer la posición exacta de cada dato dentro del arreglo mientras se itera sobre él.

Generación de arreglos

```
# Genera un arreglo con valores uniformemente espaciados dentro de un rango.
print(f'np.arange(0, 1, 0.1):\n{np.arange(0, 1, 0.1)}\n')

np.arange(0, 1, 0.1):
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]
```

EXPLICACIÓN:

En este primer ejercicio se genera un arreglo de números entre 0 y 1 (sin incluir este último), con un paso de 0.1 y al final se imprime el arreglo.

Para la generación del arreglo se hizo uso de la función **np.arange** la cual genera un arreglo de valores uniformemente espaciados dentro de un rango.

```
# Crea un arreglo nuevo lleno de ceros con una forma específica.
print(f'np.zeros((2,3,5)):\n{np.zeros((2,3,5))}\n')

np.zeros((2,3,5)):
[[[0. 0. 0. 0. 0.]
  [0. 0. 0. 0. 0.]
  [0. 0. 0. 0. 0.]]

 [[0. 0. 0. 0. 0.]
  [0. 0. 0. 0. 0.]
  [0. 0. 0. 0. 0.]]]
```

EXPLICACIÓN:

En este segundo ejercicio se genera un arreglo tridimensional de forma (2, 3, 5) lleno de ceros y lo muestra en pantalla.

Para esto se usa la función **np.zeros** la cual crea un arreglo nuevo lleno de ceros con la forma (shape) que se le pase como argumento.

```
# Crea un arreglo nuevo lleno de unos con una forma específica.
print(f'np.ones((2,3,5)):\n{np.ones((2,3,5))}\n')

np.ones((2,3,5)):
[[[1. 1. 1. 1. 1.]
  [1. 1. 1. 1. 1.]
  [1. 1. 1. 1. 1.]]

 [[1. 1. 1. 1. 1.]
  [1. 1. 1. 1. 1.]
  [1. 1. 1. 1. 1.]]]
```

EXPLICACIÓN:

Este tercer ejercicio es muy similar al anterior ya que se genera un arreglo tridimensional de forma (2, 3, 5) lleno de unos en vez de ceros y lo muestra en pantalla.

Para esto se usa la función **np.ones** la cual crea un arreglo nuevo lleno de unos con la forma (shape) que se le pase como argumento.

```
# Crea un arreglo nuevo lleno de un valor específico con una forma determinada.
print(f'np.full((2,3,4), 1.23):\n{np.full((2,3,5), 1.23)}\n')

np.full((2,3,4), 1.23):
[[[1.23 1.23 1.23 1.23 1.23]
  [1.23 1.23 1.23 1.23 1.23]
  [1.23 1.23 1.23 1.23 1.23]]

 [[1.23 1.23 1.23 1.23 1.23]
  [1.23 1.23 1.23 1.23 1.23]
  [1.23 1.23 1.23 1.23 1.23]]]
```

EXPLICACIÓN:

Este cuarto ejercicio es muy similar a los dos anteriores ya que se genera un arreglo tridimensional de forma (2, 3, 5) lleno de 1.23 en vez de ceros o de unos y lo muestra en pantalla.

Para esto se usa la función **np.full** la cual crea un arreglo nuevo con la forma (shape) que se le indique y lo llena con un valor específico que tú decidas.

```
# Crea una matriz identidad (una matriz cuadrada con unos en la diagonal y ceros en el resto).
print(f'np.identity(3):\n{np.identity(3)}\n')

np.identity(3):
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

EXPLICACIÓN:

En este quinto ejercicio generamos una matriz identidad de 3x3 (matriz cuadrada con unos en la diagonal principal y ceros en el resto de la matriz)

Para esto se usa la función **np.identity** la cual crea una matriz identidad de tamaño $n \times n$

```
# Crea una matriz identidad (similar a np.identity).
print(f'np.eye(3):\n{np.eye(3)}\n')

np.eye(3):
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

EXPLICACIÓN:

Este sexto ejercicio es muy parecido al anterior ya que generamos una matriz de 3x3 con unos en la diagonal principal y ceros en el resto de la matriz

Para esto se usa la función **np.eye** la cual genera una matriz con unos en una diagonal y ceros en el resto. Con la diferencia de que esta permite crear matrices rectangulares y mover la diagonal (k).

```
# Crea un arreglo "vacío" con una forma específica (contiene valores arbitrarios de la memoria).
print(f'np.empty((2,3,5))\n{np.empty((2,3,5))}\n')

np.empty((2,3,5))
[[[1.23 1.23 1.23 1.23 1.23]
  [1.23 1.23 1.23 1.23 1.23]
  [1.23 1.23 1.23 1.23 1.23]]
 [[1.23 1.23 1.23 1.23 1.23]
  [1.23 1.23 1.23 1.23 1.23]
  [1.23 1.23 1.23 1.23 1.23]]]
```

EXPLICACIÓN:

Este séptimo y último ejercicio generamos un arreglo 3D de forma (2,3,5) sin inicializar sus valores, mostrando números arbitrarios que estaban en la memoria.

Para esto se usa la función **np.empty** la cual crea un arreglo nuevo con la forma indicada, pero no inicializa sus valores.

Operaciones con arrays

```
# Encuentra los índices (posiciones) donde los elementos de un arreglo cumplen una condición.
vector = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
indx = np.where(vector%3 == 0)
print(f'type(indx):\t{type(indx)}')
print(f'indx:\t{indx}')

type(indx):    <class 'tuple'>
indx:  (array([2, 5, 8]),)
```

EXPLICACIÓN:

En esta sección lo que haces primero es crear un arreglo inicial con los números del 1 al 9. Para posteriormente usar la función **np.where** la cual devuelve los índices de los elementos donde la condición es verdadera.

En nuestro ejemplo devuelve una tupla con los índices de los elementos que son múltiplos de 3 en el arreglo. En este caso, los índices [2, 5, 8] corresponden a los valores [3, 6, 9].

```
# Filtra (selecciona) elementos de un arreglo usando los índices encontrados anteriormente.
filtrado = vector[indx]
print(f'filtrado:\t{filtrado}')

filtrado:    [3 6 9]
```

EXPLICACIÓN:

En esta parte lo que vamos a hacer es un filtrado, es decir vamos a seleccionar los elementos del arreglo usando los índices anteriormente encontrados. Para esto hacemos uso de **vector[indx]**, el cual usa los índices encontrados antes para extraer los elementos correspondientes del arreglo original. Aquí filtra los múltiplos de 3 y devuelve [3, 6, 9].

```
# Filtra (selecciona) elementos de un arreglo usando un arreglo de valores True/False.
vector_bool = vector%3 == 0
print(f'type(vector_bool):\t{type(vector_bool)}')
print(f'vector_bool:\t{vector_bool}')
filtrado = vector[vector_bool]
print(f'filtrado:\t{filtrado}')
```

```
type(vector_bool):    <class 'numpy.ndarray'>
vector_bool:          [False False  True False False  True False False  True]
filtrado:              [3 6 9]
```

EXPLICACIÓN:

En esta parte se muestra cómo usar indexación booleana en NumPy. Se construye un array de True/False según una condición, y luego se usa como máscara para filtrar solo los elementos que cumplen esa condición.

La condición evalúa cada elemento del arreglo y marca con True aquellos que son divisibles entre 3 (múltiplos de 3), y con False el resto.

```
# Calcula el seno de cada elemento en una lista usando una comprensión de lista
import math
opt1 = [math.sin(i) for i in [1, 2, 4]]
print(opt1)

[0.8414709848078965, 0.9092974268256817, -0.7568024953079282]
```

EXPLICACIÓN:

En esta parte recorreremos la lista [1, 2, 4], calculamos el seno de cada valor (en radianes) usando **math.sin**, y guardamos los resultados en una nueva lista mediante comprensión de listas.

```
# Calcula el seno de cada elemento en una lista y muestra cómo formatear la salida.
import math
opt1 = [math.sin(i) for i in [1, 2, 3]]
print(f'opt1: \t{opt1}')
print(f'opt1[0]: \t{opt1[0]:.30f}')

opt1:  [0.8414709848078965, 0.9092974268256817, 0.1411200080598672]
opt1[0]:      0.841470984807896504875657228695
```

EXPLICACIÓN:

En esta parte se calcula el seno de [1, 2, 3] usando de misma manera **math.sin** y demuestra cómo usar **f-strings** con formato para controlar la cantidad de decimales mostrados en la salida.

En nuestro ejemplo le pedimos que el primer elemento de la lista lo muestre con 30 decimales.

```
# Calcula el seno de cada elemento en un arreglo NumPy de manera eficiente.
opt2 = np.sin(np.array([1, 2, 3], dtype = np.uint8), dtype = np.float64)
print(f'opt2: \t{opt2}')
print(f'opt2[0]: \t{opt2[0]:.30f}')

opt2:  [0.84147098 0.90929743 0.14112001]
opt2[0]:      0.841470984807896504875657228695
```

EXPLICACIÓN:

En este ejemplo se usa **NumPy** para calcular de manera eficiente el seno de los valores [1, 2, 3] aplicando la función **np.sin** de forma vectorizada, sin necesidad de bucles. El resultado es un arreglo de tipo **float64**.

Además, se muestra cómo acceder a un elemento específico (**opt2[0]**) y formatearlo con 30 decimales para visualizar la precisión completa del cálculo.

```
# Calcula el seno solo para los elementos que cumplen una condición, guardando el resultado en otro arreglo.
x = np.array([1, 2, 3])
y = np.array([0.0, 0.0, 0.0])
np.sin(x, where = x%2 == 1, out = y)
print(f'y: {y}')

y: [0.84147098 0. 0.14112001]
```

EXPLICACIÓN:

Este ejemplo calcula el seno solo de los números impares en el arreglo `x` y almacena el resultado en `y`, mientras que para los pares mantiene su valor original (0). Esto se logra gracias a los parámetros **where (condición)** y **out (arreglo destino)** en NumPy.

```
# Suma dos arreglos elemento a elemento usando la función np.add.
x = np.array([1, 2, 3, 4])
y = np.array([4, 5, 6, 7])
z = np.add(x, y)
print(f'z: {z}')

z: [ 5  7  9 11]
```

EXPLICACIÓN:

Este código suma dos arreglos posición por posición usando **np.add**, dando como resultado un nuevo arreglo `[5, 7, 9, 11]`. Esta operación es vectorizada, por lo que NumPy la hace de forma muy rápida y eficiente, mucho más que si se sumaran con un bucle en Python.

```
# Suma dos arreglos elemento a elemento usando el operador +.
z = x + y
print(f'z: {z}')

z: [ 5  7  9 11]
```

EXPLICACIÓN:

Este código suma los arreglos `x` e `y` posición por posición usando el operador `+`. Es exactamente equivalente a `np.add(x, y)`, pero más conciso y fácil de leer.

```
# Intenta sumar arreglos con formas diferentes (incompatibles para suma directa), lo que causa un error.
try:
    print(np.array([1, 2, 3, 4]) + np.array_split([5, 6]))
except:
    print("No se pudo hacer la operación")

No se pudo hacer la operación
```

EXPLICACIÓN:

Este ejemplo demuestra que, si intentas sumar arreglos de formas incompatibles, NumPy lanza un error. Por eso, al usar `try/except`, el programa no se detiene y en su lugar muestra el mensaje "No se pudo hacer la operación".

```
# Suma un número (escalar) a cada elemento de un arreglo (broadcasting).
try:
    print(np.array([1, 2, 3, 4]) + 0.9)
except:
    print("No se pudo hacer la operación")

[1.9 2.9 3.9 4.9]
```

EXPLICACIÓN:

En este ejemplo se toma el arreglo `np.array([1, 2, 3, 4])` y se le suma el número escalar `0.9`. Gracias al **broadcasting** de NumPy, ese escalar se aplica automáticamente a cada elemento del arreglo, resultando en `[1.9, 2.9, 3.9, 4.9]`.

```
# Calcula el producto de todos los elementos de una matriz, o a lo largo de filas o columnas.
matriz = np.array([[3, 2, 4], [5, 2, 1], [8, 7, 6]])
print(f'matriz:\n{matriz}\n')
print(f'np.prod(matriz):\n{np.prod(matriz)}\n')
print(f'np.prod(matriz, axis=0):\n{np.prod(matriz, axis=0)}\n')
print(f'np.prod(matriz, axis=1):\n{np.prod(matriz, axis=1)}\n')

matriz:
[[3 2 4]
 [5 2 1]
 [8 7 6]]

np.prod(matriz):
80640

np.prod(matriz, axis=0):
[120 28 24]

np.prod(matriz, axis=1):
[ 24 10 336]
```

EXPLICACIÓN:

En este ejemplo se usa `np.prod` para calcular productos en una matriz: primero el de todos los elementos (80640), luego el de cada columna con `axis=0` ([120, 28, 24]), y finalmente el de cada fila con `axis=1` ([24, 10, 336]).

Esto muestra cómo NumPy permite operar fácilmente a nivel global o por ejes en arreglos multidimensionales.

```
# Calcula el producto acumulado de los elementos de una matriz (el producto de los elementos hasta la posición actual).
print(f'matriz:\n{matriz}\n')
print(f'np.cumprod(matriz):\n{np.cumprod(matriz)}\n')
print(f'np.cumprod(matriz, axis=0):\n{np.cumprod(matriz, axis=0)}\n')
print(f'np.cumprod(matriz, axis=1):\n{np.cumprod(matriz, axis=1)}\n')

matriz:
[[3 2 4]
 [5 2 1]
 [8 7 6]]

np.cumprod(matriz):
[ 3  6 24 120 240 240 1920 13440 80640]

np.cumprod(matriz, axis=0):
[[ 3  2  4]
 [15  4  4]
 [120 28 24]]

np.cumprod(matriz, axis=1):
[[ 3  6 24]
 [ 5 10 10]
 [ 8 56 336]]
```

EXPLICACIÓN:

En este ejemplo se usa **np.cumprod** para calcular el producto acumulado de los elementos de una matriz.

Primero genera un vector 1D con el producto progresivo de todos los valores, luego muestra el producto acumulado por columnas (**axis=0**) y finalmente por filas (**axis=1**), permitiendo ver cómo se multiplican los valores paso a paso según la dirección elegida.

```
# Calcula la media (promedio) y la desviación estándar de los elementos de una matriz.
print(f'media: {np.mean(matriz)}')
print(f'desviación estándar muestral: {np.std(matriz, ddof=1)}')
```

media: 4.222222222222222
desviación estándar muestral: 2.438123139721299

EXPLICACIÓN:

En este ejemplo se calcula la media de los elementos de una matriz con **np.mean**, obteniendo un promedio de 4.22, y la desviación estándar muestral con **np.std** usando **ddof=1**, lo que da 2.44.

Esto muestra cómo NumPy permite obtener fácilmente estadísticas descriptivas básicas de los datos.

Multiplicación Matricial

```
# Define dos arreglos que se usarán para la multiplicación matricial.
arr1 = np.array([[1,2,3], [4,5,6]]) #tamaño 2,3
arr2 = np.array([[6,5,4], [3,2,1]]) #tamaño 2,3

# Realiza la multiplicación matricial de dos arreglos usando el operador @.
print(arr1 @ arr2.T) #2,3 y 3,2 = 2,2

[[28 10]
 [73 28]]

# Realiza la multiplicación matricial de dos arreglos usando la función np.matmul.
print(np.matmul(arr1,arr2.T))

[[28 10]
 [73 28]]

# Realiza la multiplicación matricial de dos arreglos usando la función np.dot.
print(np.dot(arr1,arr2.T))

[[28 10]
 [73 28]]
```

EXPLICACIÓN:

En esta sección vemos diferentes formas de realizar la multiplicación matricial.

Primero definimos nuestros arreglos `arr1` y `arr2` de tamaño 2,3 (para que funcione la multiplicación se hace uso de la transpuesta de `arr2`).

En la primera sección se hace uso del operador `@` para indicar que se desea hacer la multiplicación de arreglos.

En la segunda se utiliza la función `np.matmul()` y le damos ambos arreglos.

Finalmente usamos la función `np.dot` e igualmente le pasamos los dos arreglos.

En los tres casos nos dan los mismos resultados, pero entre si se tienen algunas diferencias:

Caso	@	<code>np.matmul</code>	<code>np.dot</code>
Matriz 2D × Matriz 2D	Multiplicación matricial	Multiplicación matricial	Multiplicación matricial
Vector 1D × Vector 1D	Error	Escalar	Escalar (producto punto)
Matriz × Vector 1D	Error	Lo interpreta como fila/columna	Funciona, resultado válido
Arreglos con más de 2D	Sí	Sí	Funciona, pero con diferencias
Sintaxis	Más clara	Más explícita	Menos clara

```
# Intenta realizar una multiplicación matricial con tipos de datos incompatibles (causa un error).
try:
    print(0.9 @ arr1) #float y arreglo de tamaño 2,3
except:
    print("No se pudo hacer la operación")

No se pudo hacer la operación
```

Aquí se intenta multiplicar un numero decimal con una matriz usando `@`. Esto no es válido ya que `@` esta definido solo para multiplicaciones matriciales, no para escalares.


```
# Intenta realizar una multiplicación matricial con formas de arreglos incompatibles (causa un error).
try:
    print(np.array([0,9]) @ arr1) #tamaño 1 y tamaño 2,3
except:
    print("No se pudo hacer la operación")
```

➡ No se pudo hacer la operación

Se intenta multiplicar un arreglo de tamaño 1 con una matriz de tamaño 2x3. Como las dimensiones no cumplen la regla de la multiplicación de matrices (un arreglo ($m \times n$) se puede multiplicar por uno ($n \times p$) y el resultado será ($m \times p$)), ocurre el error.

```
# Realiza la multiplicación matricial de un vector fila por una matriz.
try:
    print(np.array([200, 100]) @ arr1) #tamaño 2 y tamaño 2,3
except:
    print("No se pudo hacer la operación")
```

➡ [600 900 1200]

Se multiplica un vector fila de tamaño 2 por una matriz de tamaño 2x3. La operación es válida ya que el vector fila se puede tomar como una matriz de tamaño 1x2, lo que cumple la regla de la multiplicación de matrices y da como resultado una matriz de tamaño 1x3.

```
# Realiza la multiplicación matricial de una matriz por un vector columna.
try:
    print(np.matmul(arr1, np.array([10, 20, 30]))) #2,3 y 3
except:
    print("No se pudo hacer la operación")
```

➡ [140 320]

Se multiplica una matriz de tamaño 2x3 por un vector de tamaño 3x1. La operación es válida y el resultado es un vector de tamaño 2.